

计算机组成原理第四次实验报告

920000720120 王唯嘉

2022 年 11 月 29 日

1 实验目的

1. 理解 MIPS 指令结构，理解 MIPS 指令集中常用指令的功能和编码，学会对这些指令进行归纳分类。
2. 了解熟悉 MIPS 体系的处理器结构，如延迟槽，哈佛结构的概念。
3. 熟悉并掌握单周期 CPU 的原理和设计。
4. 进一步加强运用 verilog 语言进行电路设计的能力。
5. 为后续设计多周期 cpu 的实验打下基础。

2 实验仪器

- 1、装有 Xilinx Vivado 的计算机一台
- 2、LS-CPU-EXB-002 教学系统实验箱一套

3 实验内容

单周期 CPU 就是在 1 个 CPU 周期内执行完一条指令，执行指令所需的微操作由控制器产生，结构简单，容易设计。指令系统可包括如下指令类型：

运算类指令：算术运算、逻辑运算、移位运算，如 ADD、SUB、AND、NOT、SLL、SRL 等等。

传送类指令：寄存器之间的传送、立即数传送，如 MOVZ、MOVN、MFHI 等等。

访存类指令：读存储器指令、写存储器指令，如 LB、LW、SB、SW 等等。

控制类指令：无条件转移、有条件转移，如 J、JR、JAL、BEQ 等等。

设计中所有寄存器和存储器都是异步读同步写的，即读出数据不需要时钟控制，但写入数据需时钟控制。故单周期 CPU 的运作即：在一个时钟周期内，根据 PC 值从指令 ROM 中读出相应的指令，将指令译码后从寄存器堆中读出需要的操作数，送往 ALU 模块，ALU 模块运算得到结果。如果是 store 指令，则 ALU 运算结果为数据存储的地址，就向数据 RAM 发出写请求，在下一个时钟上升沿真正写入到数据存储。如果是 load 指令，则 ALU 运算结果为数据存储的地址，根据该值从数据存 RAM 中读出数据，送往寄存器堆，根据目的寄存器发出写请求，在下一个时钟上升沿真正写入到寄存器堆中。如果非 load/store 操作，若有写寄存器堆的操作，则直接将 ALU 运算结果送往寄存器堆，根据目的寄存器发出写请求，在下一个时钟上升沿真正写入到寄存器堆中。如果是分支跳转指令，则是需要将结果写入到 pc 寄存器中的。

本实验要求：根据提供的 32 位 MIPS 框架，至少调试出 5 条指令（运算类、传送类、访存类、控制转移类等）的运行仿真波形，不需要上板。

4 实验步骤

- 1、启动 Vivado，选择 File->New Project，输入工程名称，选择工程文件的路径
- 2、选择 RTL Project，勾选 Do not specify sources at this time
- 3、在筛选器中进行如下选择：family->Artix7 package->fbg676 最后选择型号 xc7a200tfbg676-2
- 4、添加源文件 Project Manager->Add sources->Add or create design sources ->Create File，开始输入程序代码
- 5、对设计文件进行仿真，通过输入激励信号，查看仿真后的输出信号，判断是否符合设计要求。

5 实验原理

单周期 CPU 指的是一条指令的执行在一个时钟周期内完成，然后开始下一条指令的执行，即一条指令用一个时钟周期完成。之所以单周期还有一个原因：无论是寄存器堆还是内存，读操作均不受时钟沿控制，只有写操作才受上升沿控制。

数据通路：单周期 32 位 MIPS CPU 的参考如下

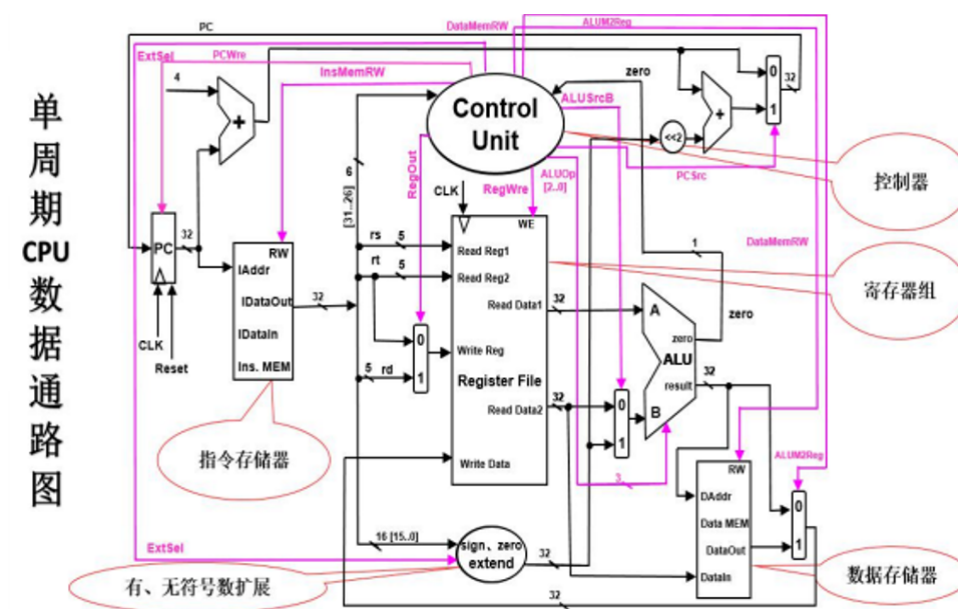


图 1: 单周期 cpu 数据通路

5.1 指令格式

指令格式：按 32 位 MIPS CPU 的指令格式 (参考 32 位 MIPS CPU 指令集规范) 如：MIPS32 指令的三种格式：

31-26	25-21	20-16	15-11	10-6	5-0
op	rs	rt	rd	sa	func

图 2: R 类型

31-26	25-21	20-16	15-0
op	rs	rt	immediate

图 3: I 类型

31-26	25-0
op	address

图 4: J 类型

其中, op: 为操作码;

rs: 为第 1 个源操作数寄存器, 寄存器地址 (编号) 是 00000 11111, 00 1F;

rt: 为第 2 个源操作数寄存器, 或目的操作数寄存器, 寄存器地址 (同上);

rd: 为目的操作数寄存器, 寄存器地址 (同上);

sa: 为位移量 (shift amt), 移位指令用于指定移多少位;

func: 为功能码, 在寄存器类型指令中 (R 类型) 用来指定指令的功能;

immediate: 为 16 位立即数, 用作无符号的逻辑操作数、有符号的算术操作数、数据加载 (Load) / 数据保存 (Store) 指令的数据地址字节偏移量和分支指令中相对程序计数器 (PC) 的有符号偏移量;

address: 为地址。

按照指令具体可以实现的功能, 我们也可以将指令进行如下划分, 分为运算类指令、传送类指令、存储器读写指令、分支指令, 这些指令都与上面的 R、I、J 类指令对应, 根据其对应的指令格式和操作码可以写出各指令的 32 位二进制数值。

运算类指令: add rd, rs, rt (说明: 以助记符表示, 是汇编指令; 以代码表示, 是机器指令)
功能: $rd \leftarrow rs + rt$ 。reserved 为预留部分, 即未用, 一般填 “0”。

传送类指令: move rd, rs 功能: $rd \leftarrow rs$ 。

存储器读/写指令: sw rt, immediate(rs) 写存储器功能: $memory[rs + (\text{sign-extend})immediate] \leftarrow rt$;
immediate 符号扩展再相加。

分支指令: beq rs, rt, immediate 功能: $\text{if}(rs=rt) \text{ pc} \leftarrow \text{pc} + 4 + (\text{sign-extend})immediate \ll 2$;

6 实现代码

指令寄存器部分代码实现

```
“timescale 1ns / 1ps
```

```
module inst_rom(
```

```
    input      [4 :0] addr, // 指令地址
```

```
    output reg [31:0] inst    // 指令
```

```
);
```

```
    wire [31:0] inst_rom[19:0]; // 指令存储器, 字节地址 7'b000_0000~7'b111_1111
```

```
//----- 指令编码 -----/指令地址/--- 汇编指令 -----/- 指令结果 -----
```

```
    assign inst_rom[ 0] = 32'h24010002; // 寄存器 0 与立即数 2 相加放寄存器 1
```

```
    assign inst_rom[ 1] = 32'h24020003; // 寄存器 0 与立即数 3 相加放寄存器 2
```

```

assign inst_rom[ 2] = 32'h00411821; // 寄存器1和2相加放寄存器3
assign inst_rom[ 3] = 32'hac030000; //将寄存器3的值放入地址0
assign inst_rom[ 4] = 32'h08000006; //跳转到 pc=00000018
assign inst_rom[ 6] = 32'h00032040; //将寄存器3逻辑左移一位放到寄存器4
//读指令,取4字节
always @(*)
begin
    case (addr)
        5'd0 : inst <= inst_rom[0 ];
        5'd1 : inst <= inst_rom[1 ];
        5'd2 : inst <= inst_rom[2 ];
        5'd3 : inst <= inst_rom[3 ];
        5'd4 : inst <= inst_rom[4 ];
        5'd6 : inst <= inst_rom[6 ];
        default: inst <= 32'd0;
    endcase
end
endmodule

```

仿真 tb 部分代码实现

```

“timescale 1ns / 1ps

module tb;

    // Inputs
    reg clk;
    reg resetn;
    reg [4:0] rf_addr;
    reg [31:0] mem_addr;

    // Outputs
    wire [31:0] rf_data;
    wire [31:0] mem_data;
    wire [31:0] cpu_pc;
    wire [31:0] cpu_inst;

    // Instantiate the Unit Under Test (UUT)
    single_cycle_cpu uut (
        .clk(clk),
        .resetn(resetn),
        .rf_addr(rf_addr),
        .mem_addr(mem_addr),

```



```
.rf_data(rf_data),
.mem_data(mem_data),
.cpu_pc(cpu_pc),
.cpu_inst(cpu_inst)
);

initial begin
    // Initialize Inputs
    clk = 0;
    resetn = 0;
    rf_addr = 0;
    mem_addr = 0;
    // Wait 100 ns for global reset to finish
    #100;
    resetn = 1;
    //100s
    rf_addr=1;//寄存器1中应为2
    #10;
    rf_addr=2;//寄存器2中应为3
    #10;
    rf_addr=3;//寄存器3中应为5
    #10;
    //存储器0中应为5
    #10;
    //pc应为00000018
    #10;
    rf_addr=4;//寄存器4中应为10
    #10;

    // Add stimulus here
end
always #5 clk=~clk;
endmodule
```

7 实验结果记录

执行指令后的仿真波形图如下：

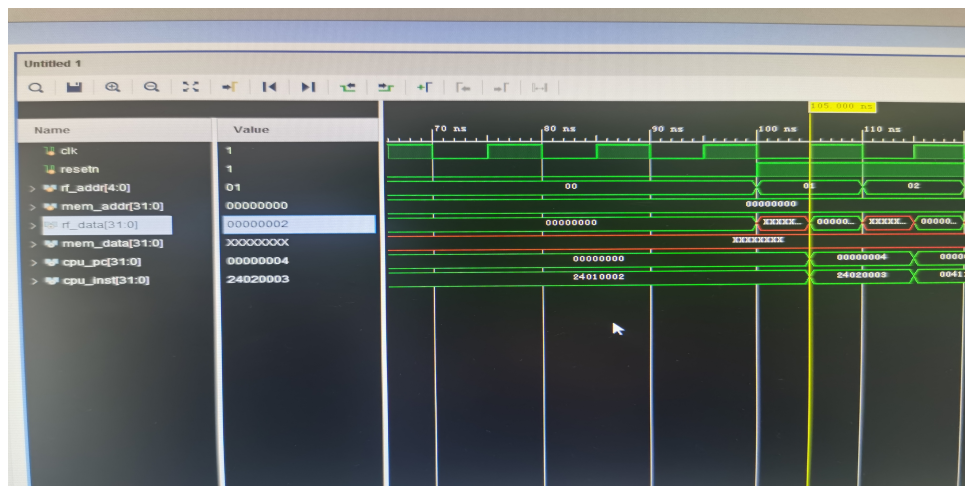


图 5: 第一条指令执行后结果

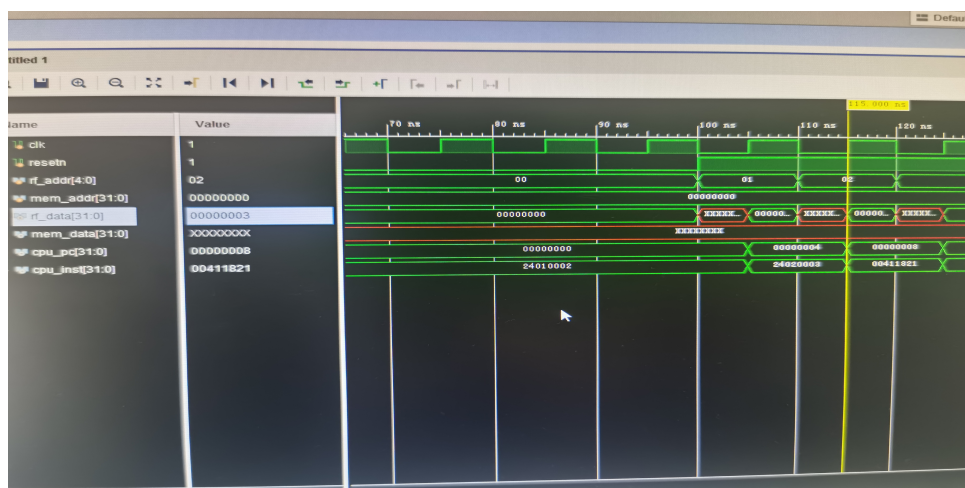


图 6: 第二条指令执行后结果

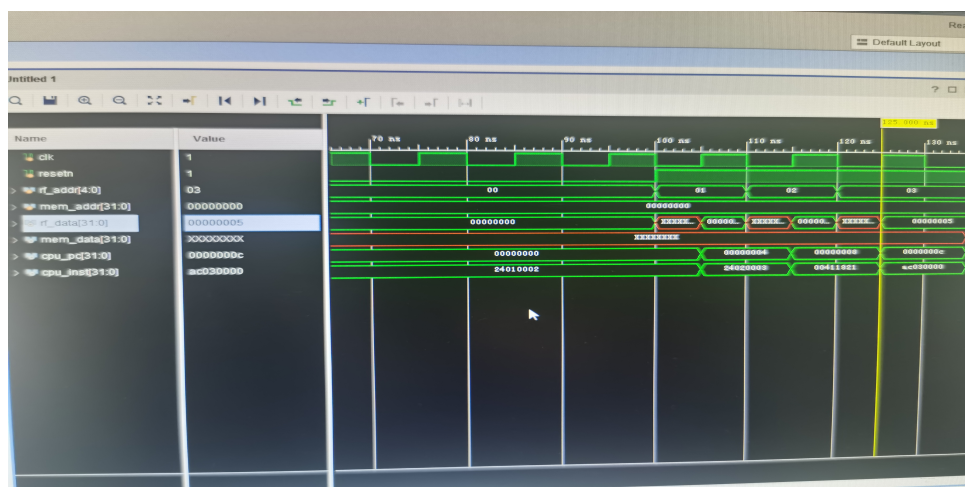


图 7: 第三条指令执行后结果

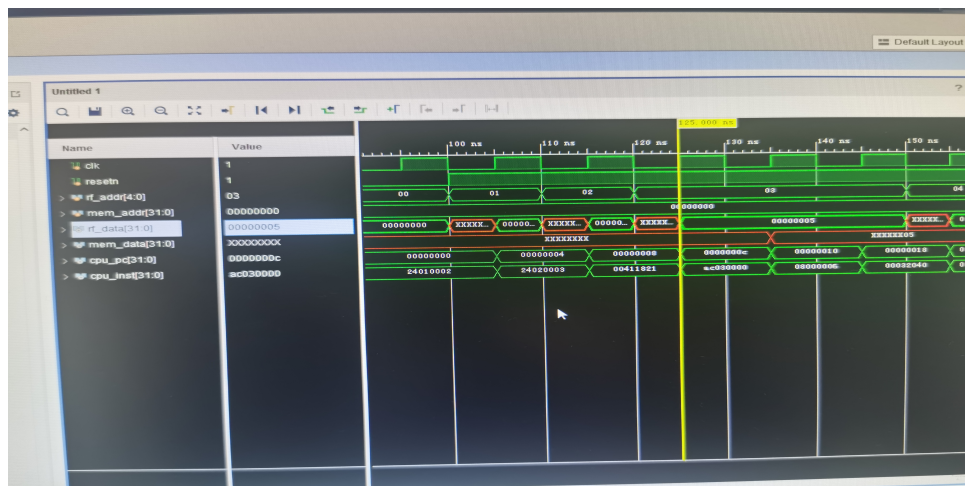


图 8: 第四条指令执行后结果

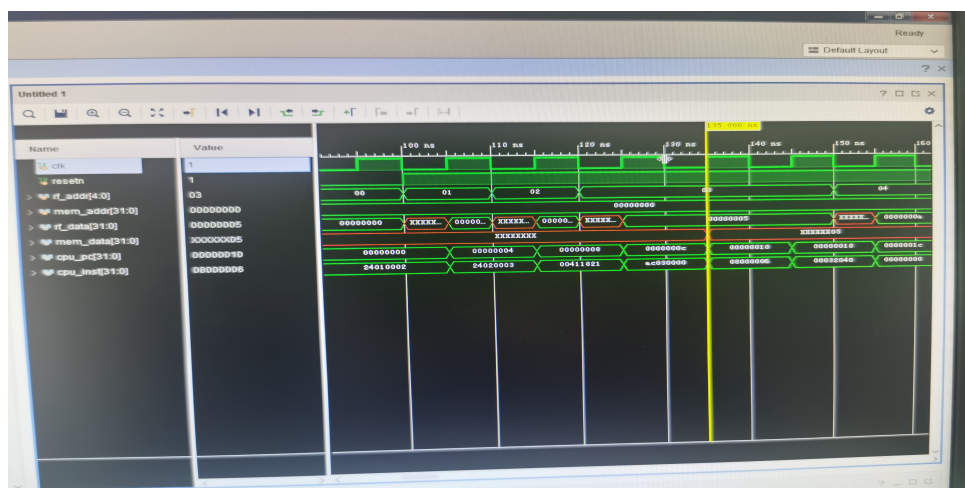


图 9: 第五条指令执行后结果

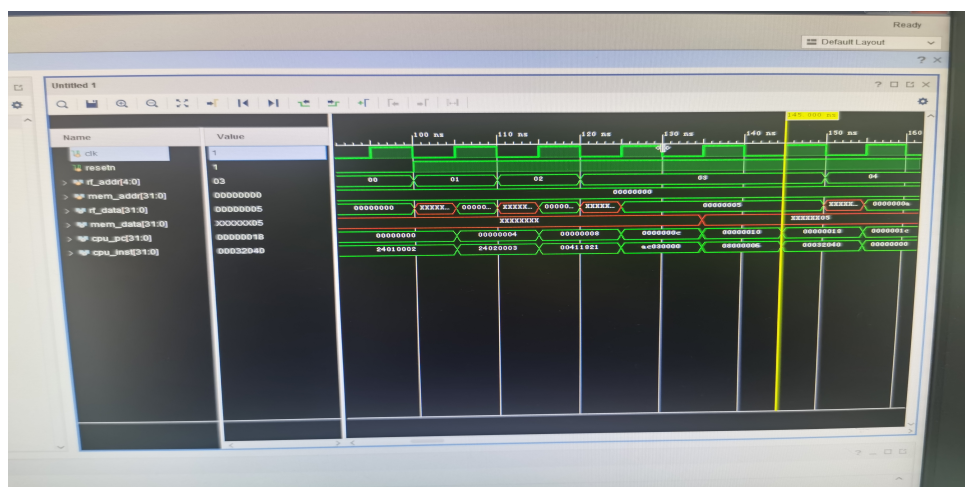


图 10: 第六条指令执行后结果

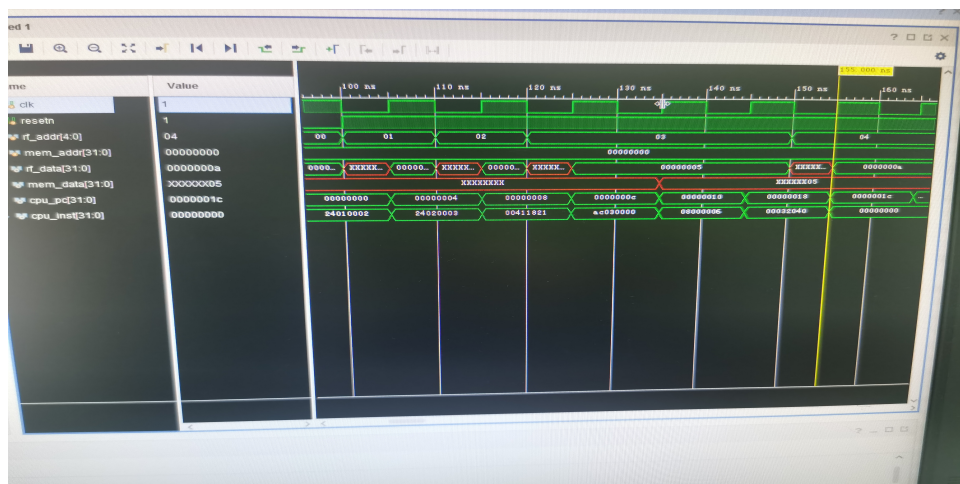


图 11: 第七条指令执行后结果

从第一张仿真波形图可以看到第一条指令是将寄存器 0 与立即数 2 相加送到寄存器 1，此时寄存器 1 为 2；第二条指令是将寄存器 0 与立即数 2 相加送到寄存器 2，此时寄存器 2 为 3；第三条指令是将寄存器 1 与寄存器 2 相加送到寄存器 3，此时寄存器 3 为 5；第四条指令是将寄存器 3 的内容送到存储器地址 0 中，此时存储器地址 0 中的值为 5；第五条指令是跳转指令，可以看出 PC 值从 10 跳转到了 pc 值为 18；第六条指令是将寄存器 3 中的内容左移一位放到寄存器 4，此时寄存器 4 中的内容为 a。